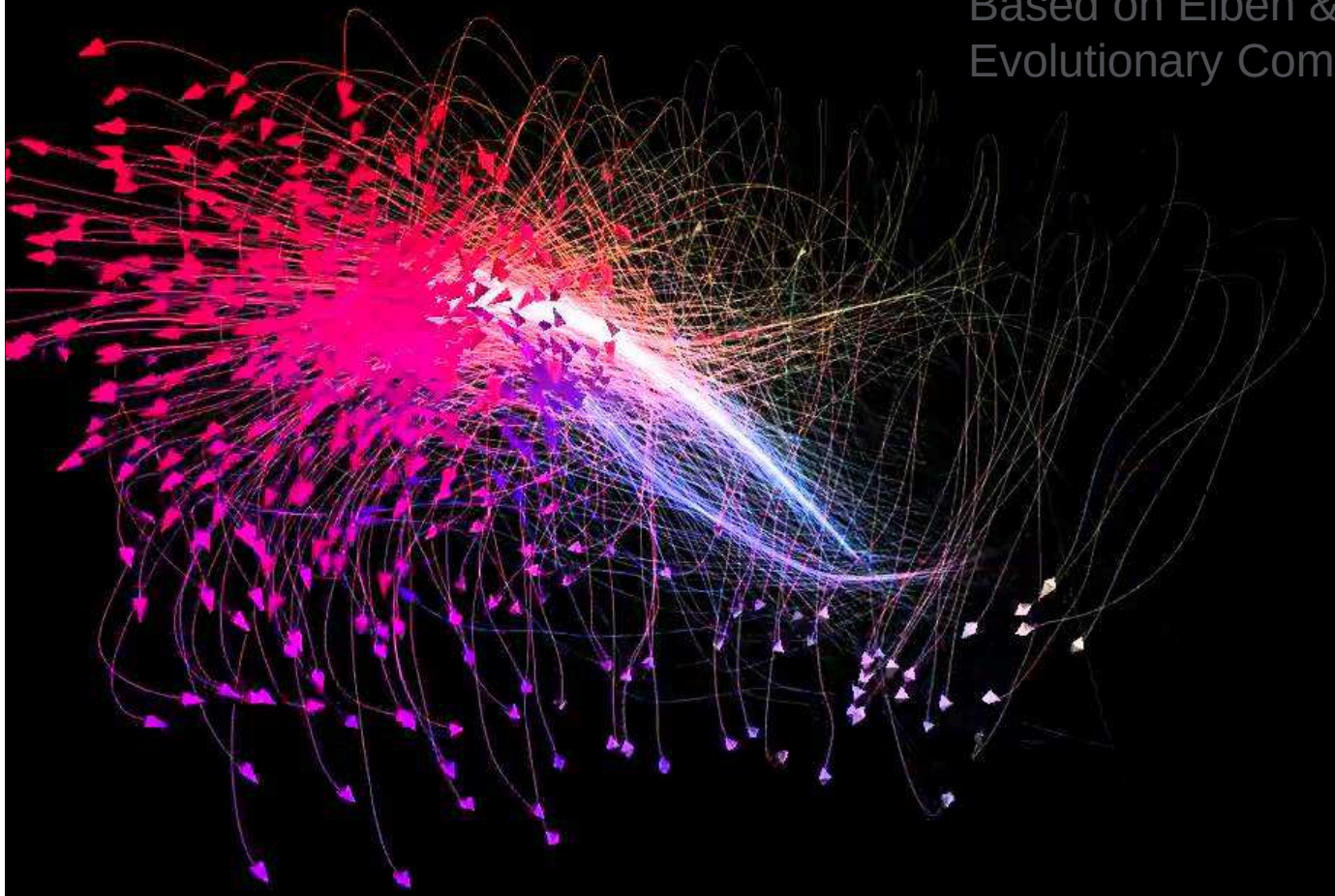
Computational Intelligence

Evolutionary computing

Based on Eiben & Smith: Introduction to Evolutionary Computing, Chapters 2,3



Anikó Ekárt & Alina Patelli

Aston University

Outline

By the end of this lecture you should be able to

- … explain the inspiration for evolutionary computing
- … understand the main principles of evolutionary algorithms
- … design and implement the key elements of an evolutionary algorithm:
  - ✓Selection
  - ✓Variation operators
  - ✓Generations

    (Example: TSP)



The inspiration for Evolutionary Computing



Historical perspective

1948, Turing:

proposes “genetical or evolutionary search”

1962, Bremermann:

optimisation through evolution and recombination

1964, Rechenberg:

introduces evolution strategies

1965, Fogel, Owens and Walsh:

introduce evolutionary programming

1975, Holland:

introduces genetic algorithms

1985, Cramer:

introduces tree-based genetic programming

1992, Koza:

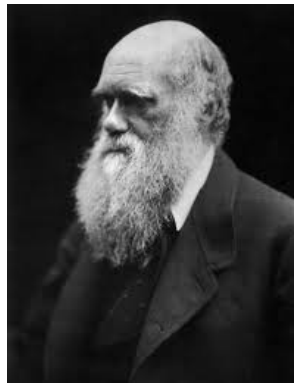
expands genetic programming

Now:

3 major & 10+ smaller conferences, 4 journals, 1000+ papers published / year



Darwinian evolution:
survival of the fittest



- All environments have finite resources (i.e., can only support a limited number of individuals)
- Life forms have basic instinct/ lifecycles geared towards reproduction, therefore some kind of **selection** is inevitable
- Those individuals that compete for the resources most effectively have increased chance of reproduction
- Note: fitness in natural evolution is a derived measure

*"It is not the strongest of the species that survive, nor the most intelligent, but the one most responsive to **change**."*

Charles Darwin (1802-1889)

Darwinian evolution: Diversity drives change

- Phenotypic traits:
  - ✓ Behaviour / physical differences that affect response to environment
  - ✓ Partly determined by inheritance, partly by factors during development
  - ✓ Unique to each individual, partly as a result of random changes

- If phenotypic traits
  lead to higher chances of reproduction
  can be inherited
  then they will tend to increase in subsequent generations, leading to new combinations of traits …



Darwinian evolution: what to mimic?

- A **population** consists of a diverse set of **individuals**
- Combinations of traits that are better adapted tend to spread in population

  Individuals are **“units of selection”**
- Variations occur through **random changes** (permanent source of diversity), coupled with **selection**:

  Population is the **“unit of evolution”**

  Note the absence of “guiding force”

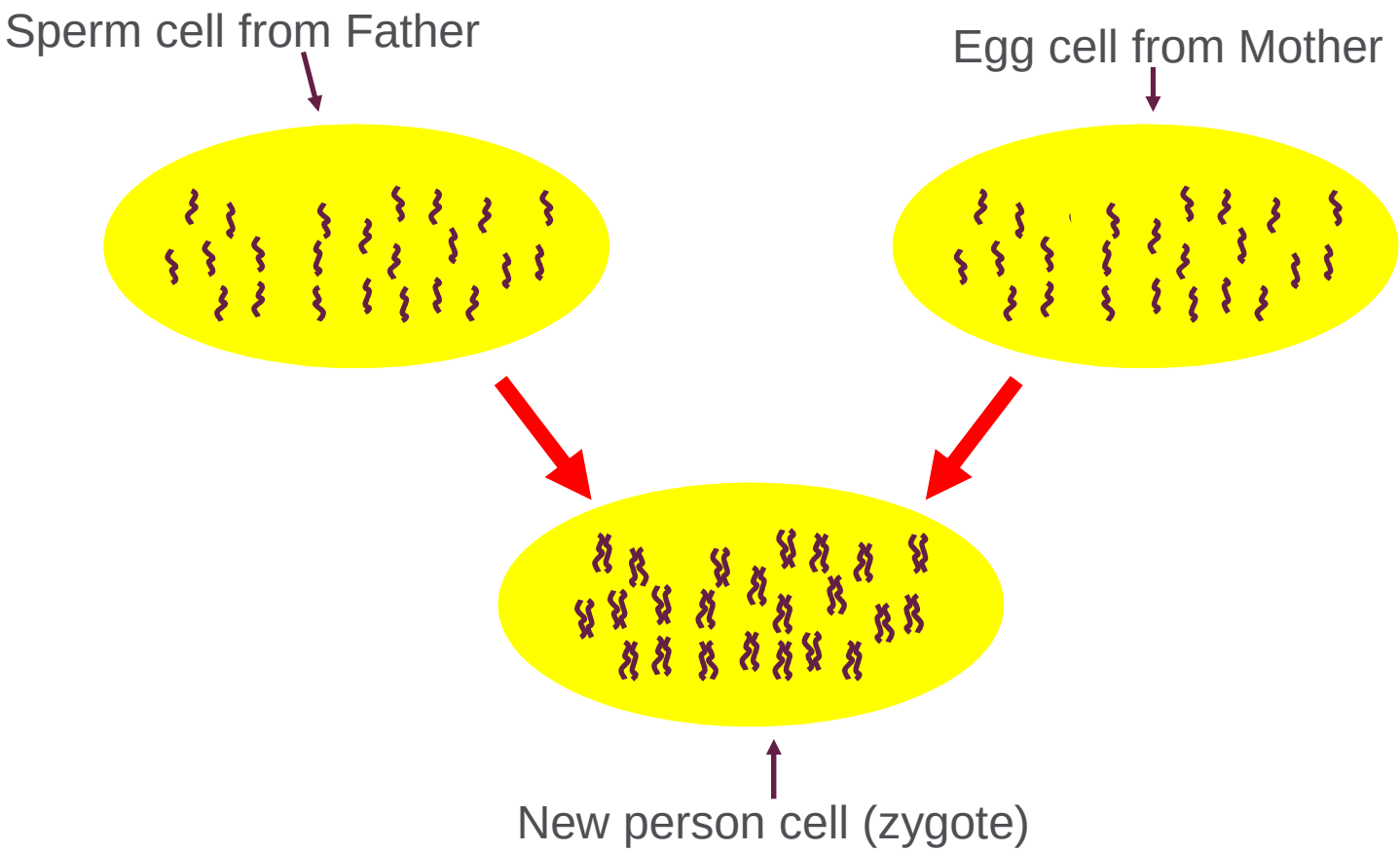


Genetics

- The information required to build a living organism is coded in the DNA of that organism
- Genotype (DNA inside) determines phenotype
- Genes → phenotypic traits is **complex**
- Small changes in the genotype lead to small changes in the organism (e.g., height, hair colour)
- Genes are encoded in strands of DNA called chromosomes
- In most cells, there are two copies of each chromosome
- The complete genetic material in an individual's genotype is called the Genome



Genetics: Fertilisation

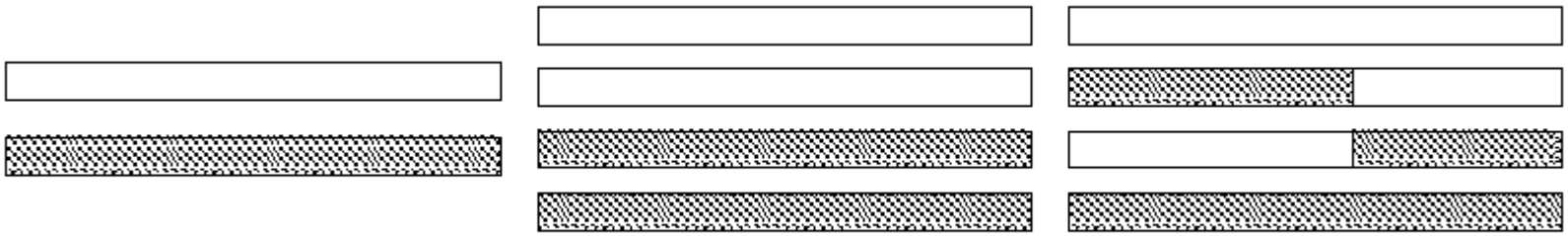


Human body cells contains 23 pairs of chromosomes

After fertilisation, the new zygote rapidly divides resulting in many cells all with the same genetic contents

Genetics: Crossing-over during meiosis

- Chromosome pairs align and duplicate
- Inner pairs link at a *centromere* and swap parts of themselves



- Outcome is one copy of maternal/paternal chromosome plus two entirely new combinations

Genetics: Mutation

- Occasionally some of the genetic material changes very slightly during this process (replication error)
- This means that the child might have genetic material information not inherited from either parent
- This can be
  - ✓ catastrophic: offspring in not viable (most likely)
  - ✓ neutral: new feature does not influence fitness
  - ✓ advantageous: strong new feature occurs
- Redundancy in the genetic code forms a good way of error checking



Motivation for evolutionary computing

- Nature has always served as a source of inspiration for engineers and scientists
- The best problem solver known in nature is:
  - ✓ the (human) brain → neurocomputing
  - ✓ the evolution mechanism that created the human brain (after Darwin's Origin of Species)

    → **evolutionary computing**
- ROBUST PROBLEM SOLVING technology needed for complex problems



The EC metaphor

- A **population** of **individuals** exists in an environment with limited resources
- ***Competition*** for those resources causes **selection** of those ***fitter*** individuals that are better adapted to the environment
- These individuals act as seeds for the **generation** of new individuals through **recombination** and **mutation**
- The new individuals have their **fitness evaluated** and compete (possibly also with parents) for survival
- Over time ***natural selection*** causes a rise in the fitness of the population



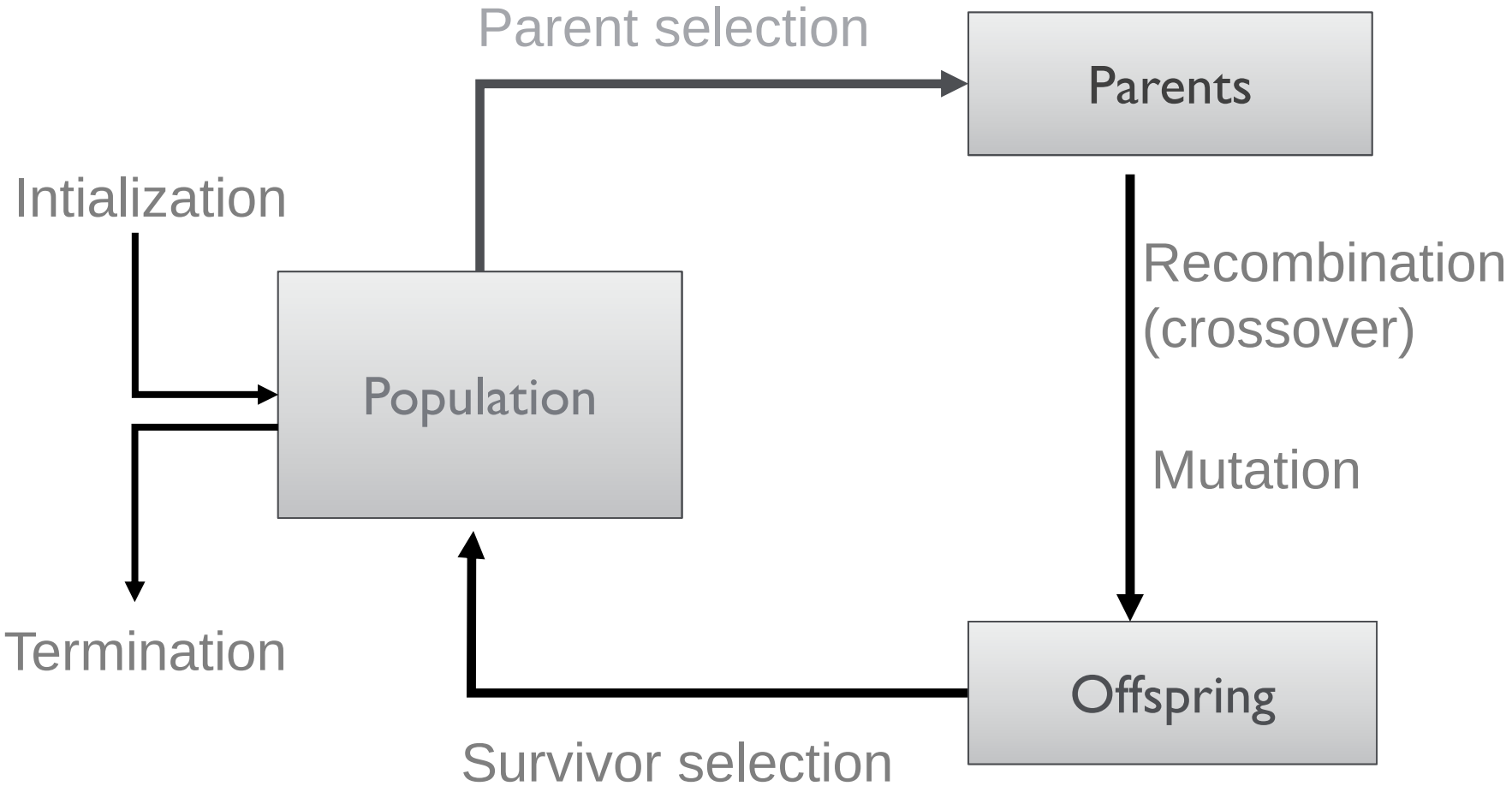
	Natural evolution	Artificial evolution
Fitness	Observed, effect of selection	Predefined
Selection	Complex, based on environment, other individuals of the same species and other species	Randomised operator, based on fitness values
Genotype-phenotype mapping	Highly complex, influenced by environment	Relatively simple mathematical transformation
Variation	Offspring from one or two parents	Offspring from one, two or more parents
Execution	Parallel, decentralised, not synchronised	Typically centralised, with synchronised birth and death
Population	Structured populations of varying size according to births and deaths	Typically unstructured and panmictic, of constant size



The main principles of evolutionary algorithms



General working of evolutionary algorithms



The pseudocode

```
BEGIN
  INITIALISE population with random candidate solutions;
  EVALUATE each candidate;
  REPEAT UNTIL ( TERMINATION CONDITION is satisfied ) DO
    1 SELECT parents;
    2 RECOMBINE pairs of parents;
    3 MUTATE the resulting offspring;
    4 EVALUATE new candidates;
    5 SELECT individuals for the next generation;
  OD
END
```

A generation

Common model of evolutionary processes

- Population of individuals
- Individuals have a **fitness**
- **Variation** operators: crossover, mutation
- **Selection** towards higher fitness
  - ✓ “survival of the fittest” and
  - ✓ “mating of the fittest”

Neo Darwinism:

Evolutionary progress towards higher life forms

=

Optimisation according to some fitness measure



Main components of an evolutionary algorithm

- Representation
- Fitness evaluation
- Selection
- Variation operators
- Initialisation and termination

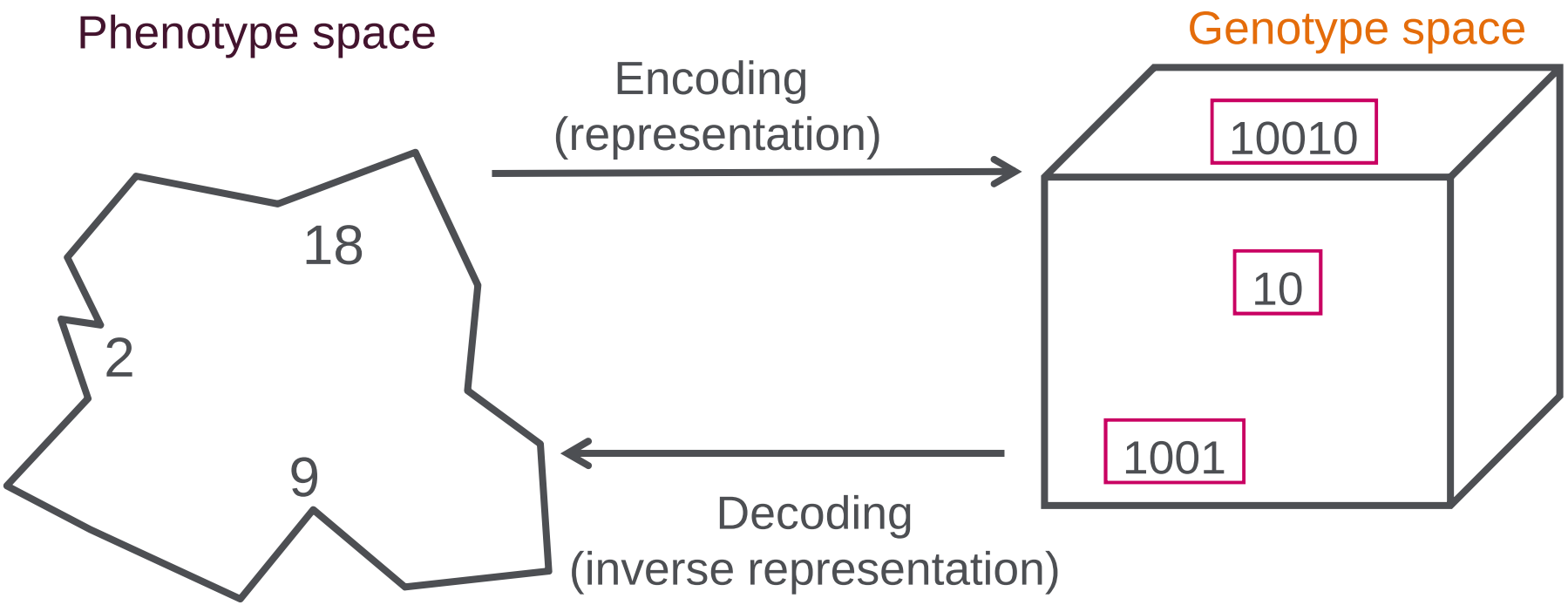


Representation

- Role: provides code for candidate solutions that can be manipulated by variation operators
- Two levels of existence
  - phenotype: object in original problem context
  - genotype: code to denote that object (chromosome)
- Chromosomes contain genes, which are in (usually fixed) positions called loci (sing. locus) and have a value (allele)



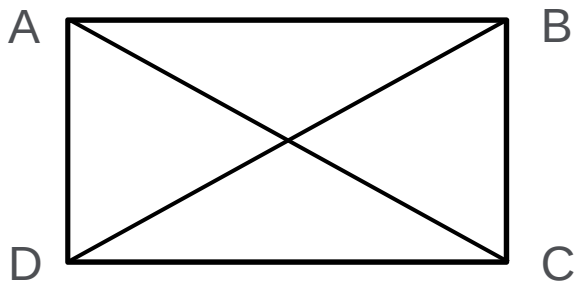
Representation example



- Integers in binary representation

Representation of TSP

- The list of cities in the order to be visited, possibly each city decoded by an integer number (permutation)
- The list of positions in the tour for each city, recorded in a specific order (could be alphabetical order of names)



Do we have to restrict all tours to start (and finish) at A?

Tour	A B C D	A D B C
Representation 1	1 2 3 4	1 4 2 3
Representation 2	1 2 3 4	1 3 4 2

Evaluation (fitness) function

- Role:
  - ✓ Represents the task to solve, the requirements to adapt to (can be seen as “the environment”)
  - ✓ Enables selection (provides basis for comparison)
  - ✓ e.g., some phenotypic traits are advantageous, desirable, e.g. big ears cool better, these traits are rewarded by more offspring that will expectedly carry the same trait
- A.k.a. *quality* function or *objective* function
- Assigns a single real-valued fitness to each phenotype which forms the basis for selection
- Typically we talk about fitness being maximised



Evaluation for TSP

- The length of the tour is the fitness value

- The genotype is transformed into the phenotype
- The distances between consecutive cities are retrieved / calculated
- The distances are summed up

- This is a minimisation problem (finding the shortest tour)



Population

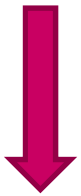
- Role: holds the candidate solutions of the problem as individuals (genotypes)
- Formally, a population is a multiset of individuals, i.e. repetitions are possible
- Population is the basic unit of evolution, i.e., the population is evolving, not the individuals
- **Selection operators** act on **population** level
- **Variation operators** act on **individual** level
- **Diversity** of a population can refer to the number of different fitness values or phenotypes or genotypes present



Two competing forces in evolution

Increasing population **diversity**
by variation operators

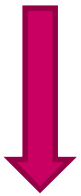
- mutation
- recombination



Novelty

Decreasing population **diversity**
by selection

- of parents
- of survivors



Quality



What is selection and how do we implement it?



Selection

Role:

- Identifies individuals
  - to become parents
  - to survive
- Pushes population towards higher fitness
- Usually probabilistic
  - High quality solutions more likely to be selected than low quality
  - But not guaranteed
  - All individuals in current population usually have non-zero probability of being selected
- This helps escape from local optima
- ***Note:*** selection does not depend on representation



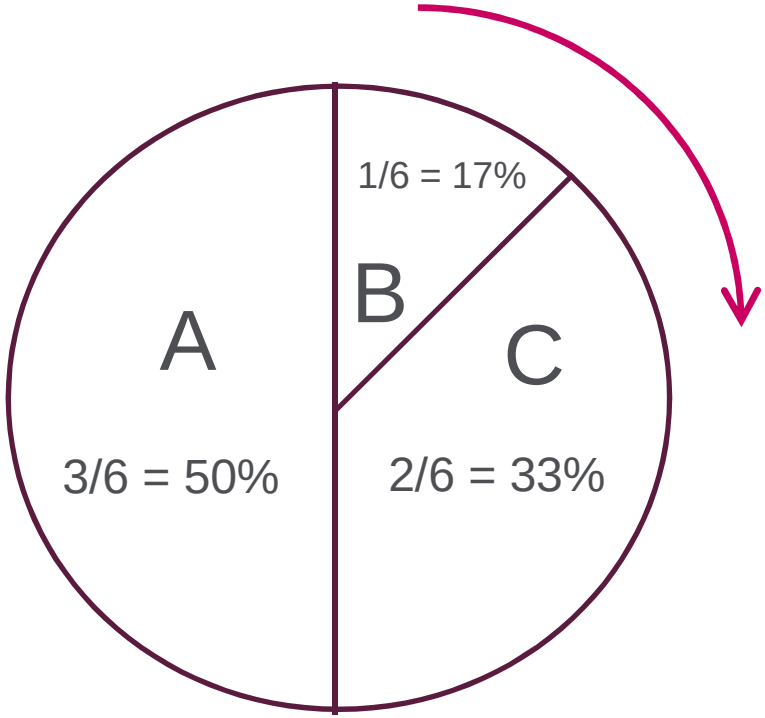
Parent selection by Roulette wheel



fitness(A) = 3

fitness(B) = 1

fitness(C) = 2



Also known as fitness proportionate selection

Probability for individual *i* to be selected for mating in a population size μ

$$P_{FPS}(i) = f_i \Big/ \sum_{j=1}^{\mu} f_j$$

Parent selection by tournament



- Does not rely on global population statistics
- Can be used when there is no external fitness function e.g. evolving game players

- This method uses only local fitness information:
  - Pick *k* members at random then select the best of these (can be deterministic or with some probability)
  - Repeat to select more individuals

- The probability of selecting an individual will depend on its rank and *k* (higher is more selective)



Parent selection: uniform

$$P_{uniform}(i) = \frac{1}{\mu}$$

- Parents are selected by uniform random distribution whenever an operator needs one/some
- Uniform parent selection is unbiased - every individual has the same probability to be selected



Survivor selection (replacement)

- Most EAs use fixed population size so need a way of going from (parents + offspring) to next generation
- From a set of μ parents and λ offspring, a set of μ individuals forming the next generation are selected
- The parent selection mechanisms can also be used for selecting survivors
- Survivor selection can be divided into two approaches:
  - Age-Based Selection
    - Fitness is not taken into account
  - Fitness-Based Replacement
  - Mixed is also possible



Fitness-based replacement

Elitism: always keep at least one copy of the fittest solution so far

Specific for evolution strategies and evolutionary programming:

- (μ,λ)-selection
  - based on the set of children only ($\lambda > \mu$)
  - choose best μ

- $(\mu+\lambda)$-selection
  - based on the set of parents and children
  - choose best μ

- $\lambda \approx 3 \bullet \mu$ seems popular lately



Population management

- Generational model (age-based)
  - each individual survives for exactly one generation
  - the entire set of parents is replaced by the offspring
- Steady-state model (mixed)
  - one offspring is generated per generation
  - one member of population is replaced
- Genetic algorithms and genetic programming specific



What are variation operators and how do they work?



Variation operators

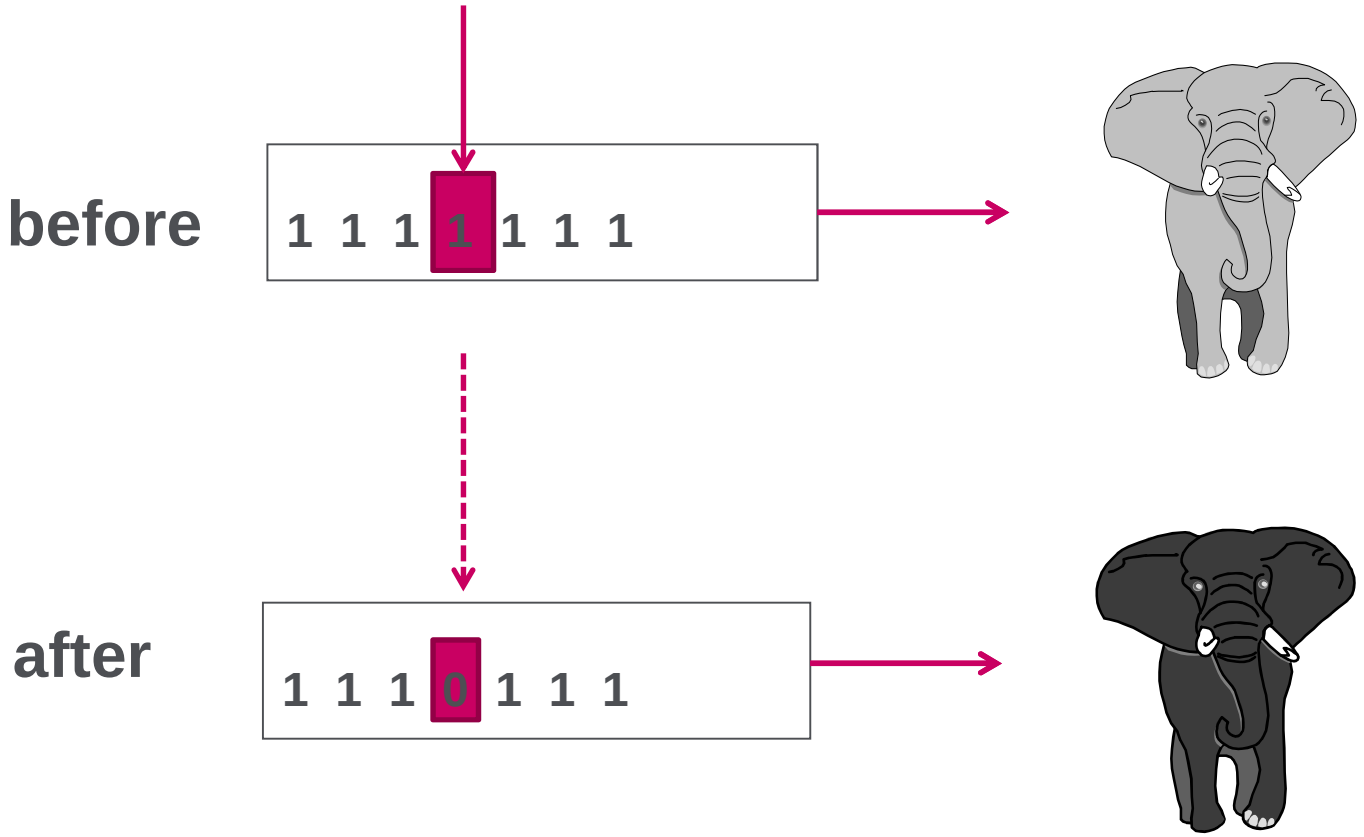
- Role: to generate new candidate solutions
- Mutation takes one input and produces one output
- Recombination takes more than one input
  - Crossover takes two input and typically produces two output
- There has been much debate about relative importance of recombination and mutation
  - Nowadays most EAs use both
  - Variation operators must match the given representation



Mutation

Role: causes **small**, **random** variance

Acts on one genotype and delivers another

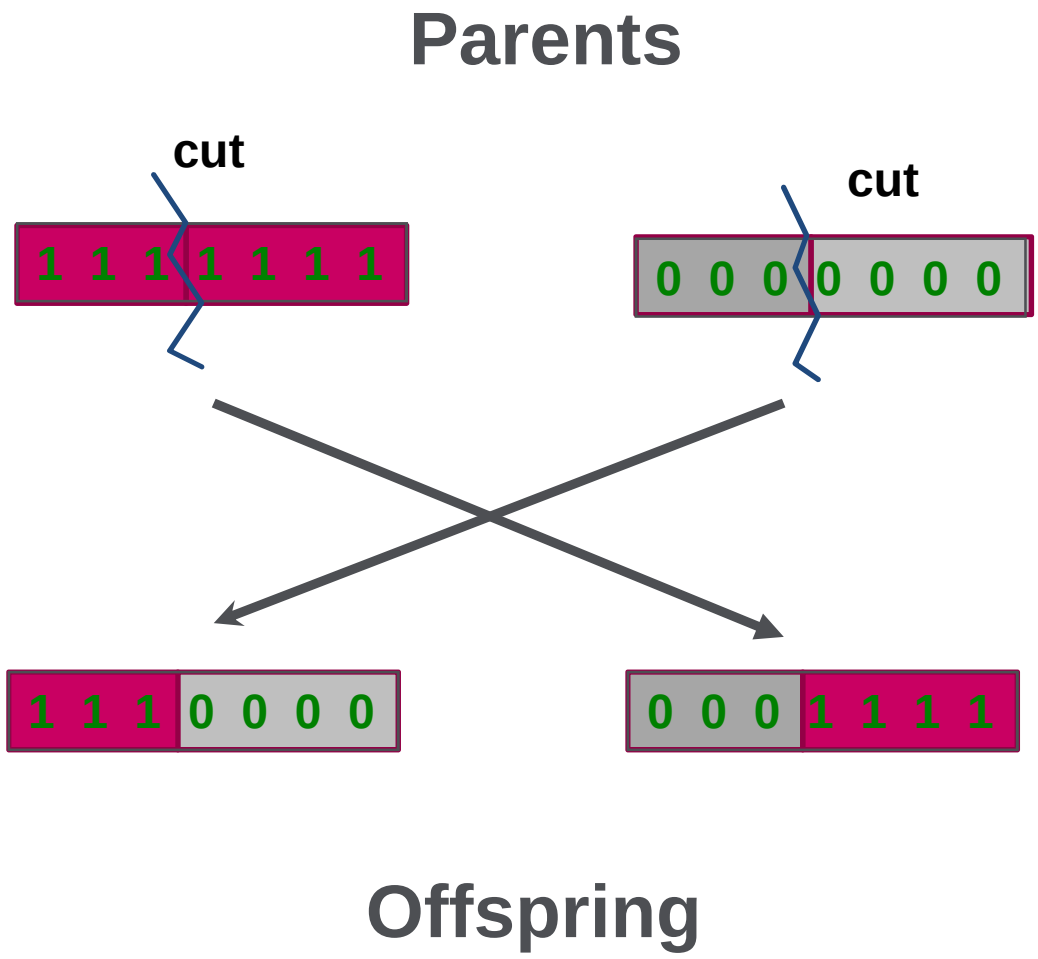


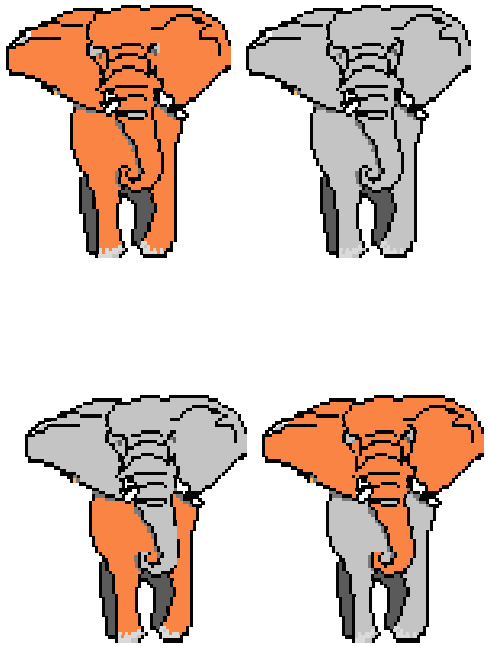
Recombination

- Role: merges information from parents into offspring
- Choice of what information to merge is stochastic
- Most offspring may be worse, or the same as the parents
- Hope is that some are better by combining elements of genotypes that lead to good traits
- Principle has been used for millennia by breeders of plants and livestock



Recombination: one point crossover





How do we design variation operators in practice?



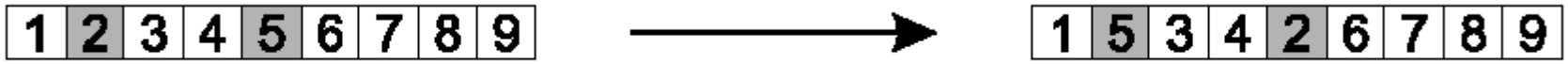
Mutation for TSP

- Normal mutation operator:

  Change one city to another…would this work?

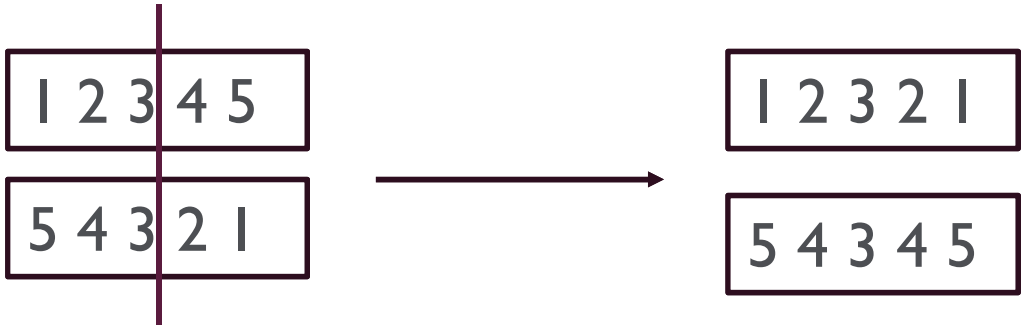
  ✕ Leads to inadmissible solutions!

- Therefore must change at least two values
- Mutation parameter now reflects the probability that some operator is applied once to the whole string
- Swap mutation:



Crossover operators for TSP

- “Normal” crossover operators will often lead to inadmissible solutions

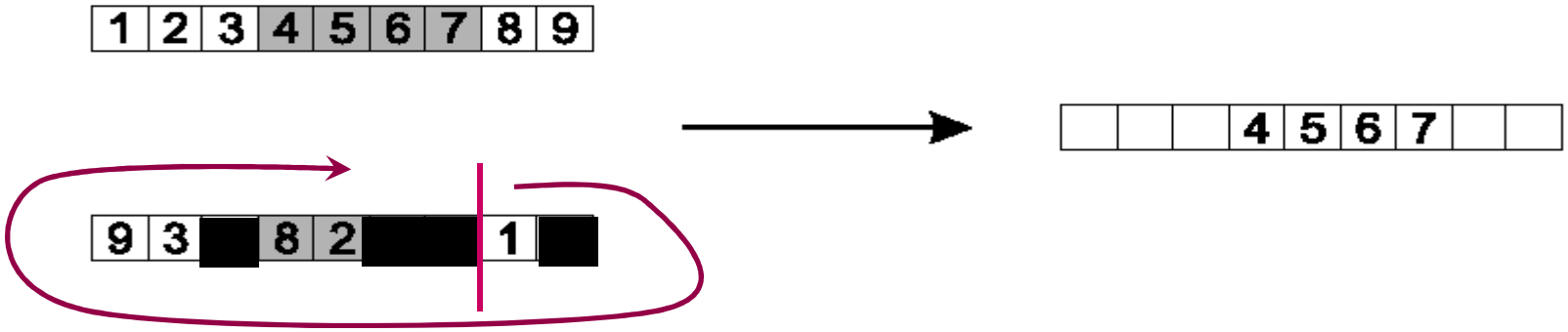


- Many specialised operators have been devised which focus on combining order or adjacency information from the two parents

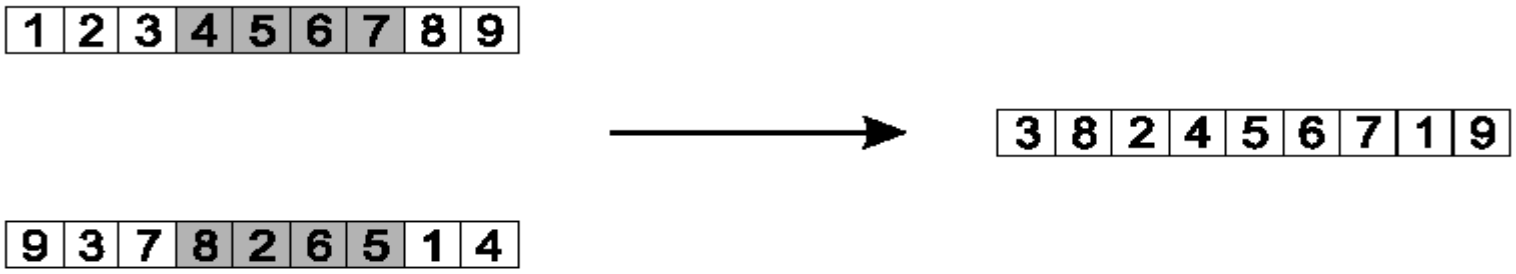


Order 1 crossover for TSP

- Copy randomly selected list from first parent



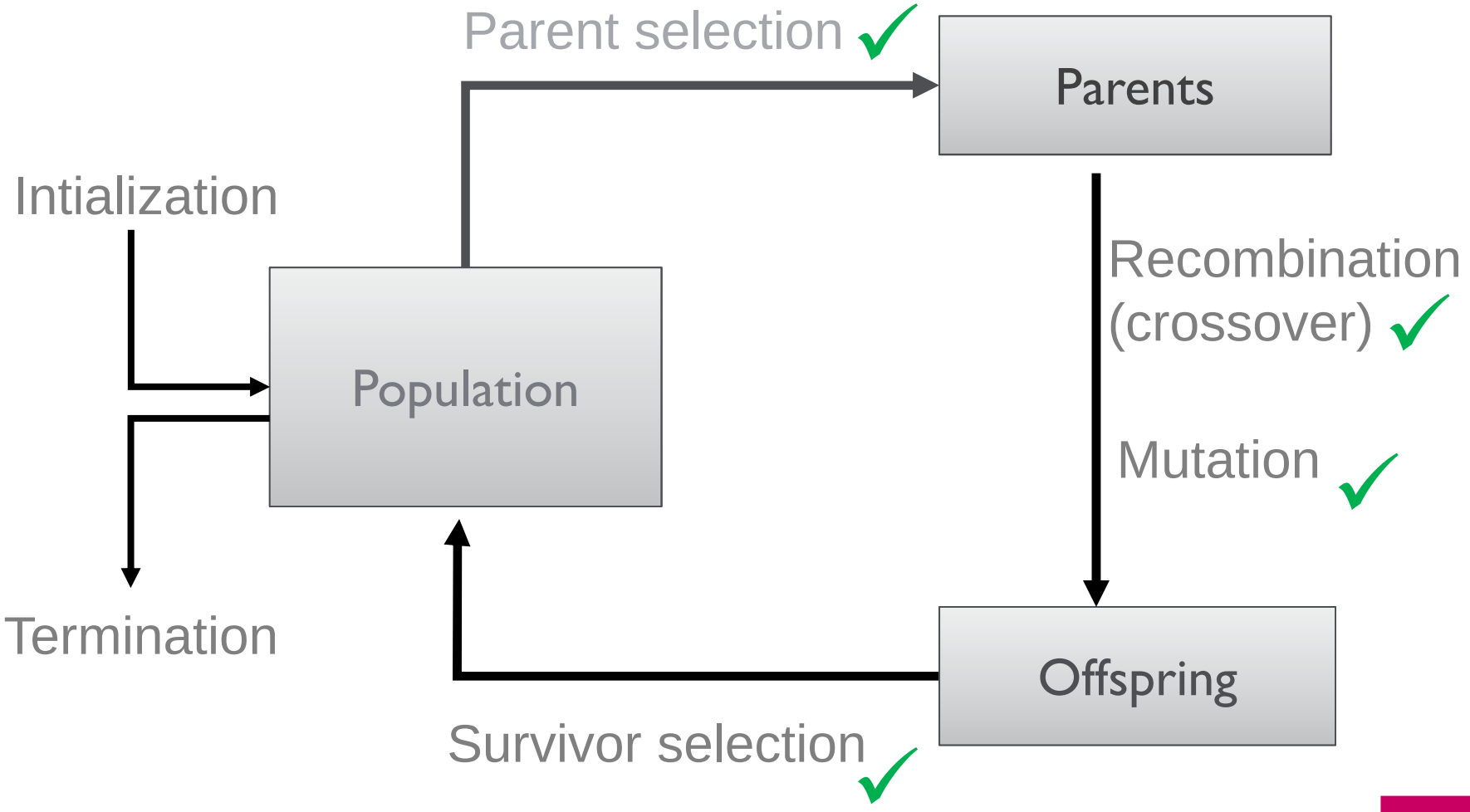
- Copy rest from second parent in order 1,9,3,8,2



How do we put it all together into an evolutionary algorithm?



General working of evolutionary algorithms



Initialisation / Termination

- Initialisation usually done at random
  - Need to ensure even spread and mixture of possible allele values
  - Can include existing solutions, or use problem-specific heuristics, to “seed” the population
- Termination condition checked in every generation
  - Reaching some (known / desired) fitness
  - Reaching some maximum allowed number of generations
  - Reaching some minimum level of diversity
  - Reaching some specified number of generations without fitness improvement



Example set of operators & parameters (TSP)

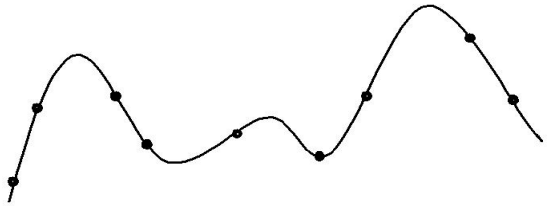
Representation	Permutation
Recombination	Order 1 crossover
Recombination probability	100%
Mutation	Swap
Mutation probability	70%
Parent selection	Tournament, size 2
Survivor selection	Elitism & generational
Population size	100
Number of offspring	2
Initialisation	Random
Termination	100 generations
Fitness evaluation	length of tour, minimise



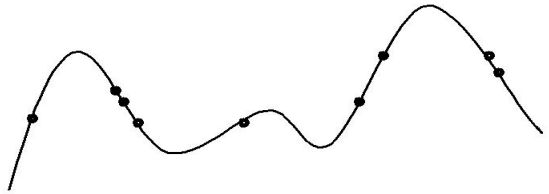
What can we expect when we run an evolutionary algorithm?



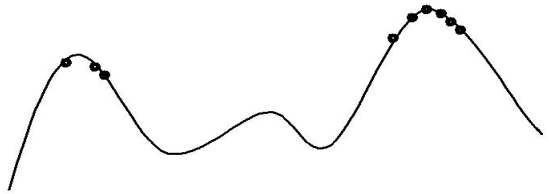
Typical evolutionary algorithm behaviour



Early stage:
quasi-random population distribution



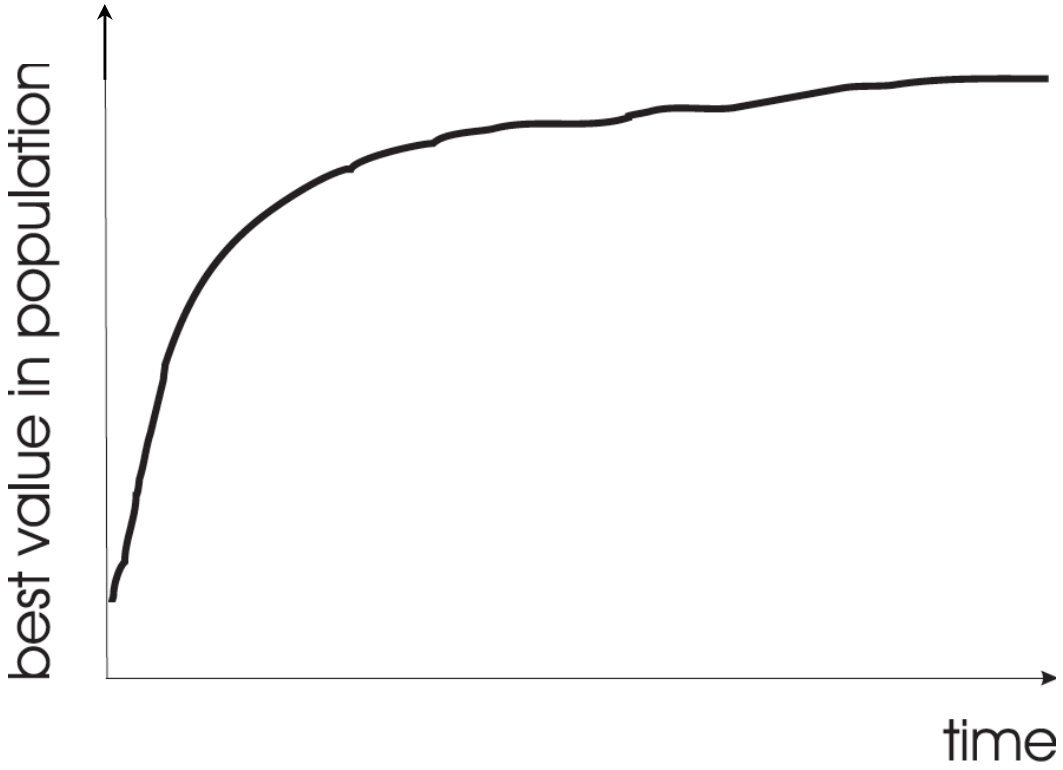
Mid-stage:
population arranged around/on hills



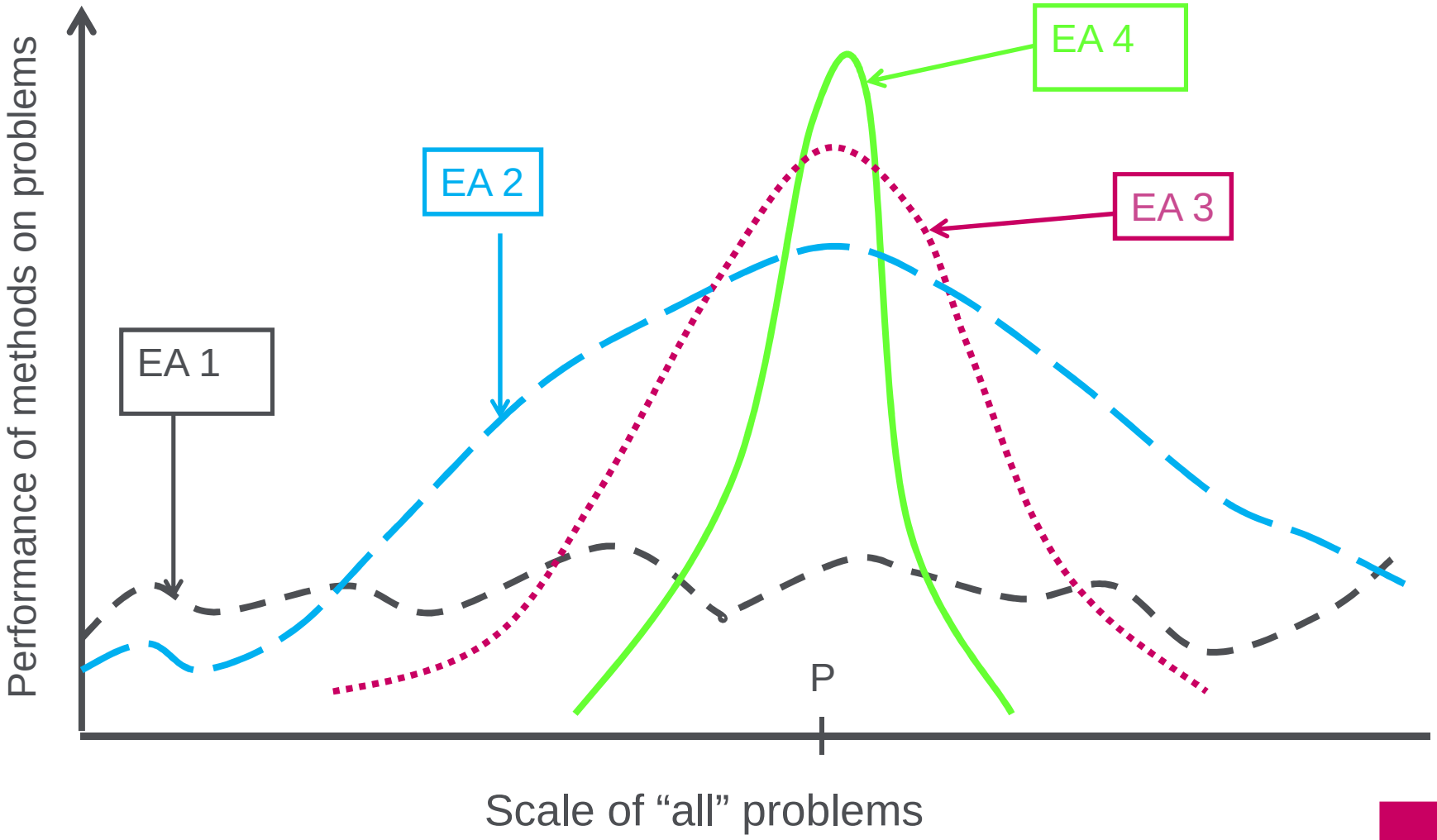
Late stage:
population concentrated on high hills



Typical EA behaviour:
Typical run: progression of fitness



Typical EA behaviour:
EAs as problem solvers: Michalewicz view (1996)



The different types of evolutionary algorithms

- Historically different flavours of EAs have been associated with different data types to represent solutions
  - Binary strings : Genetic Algorithms
  - Real-valued vectors : Evolution Strategies
  - Finite state Machines: Evolutionary Programming
  - LISP trees: Genetic Programming

- These differences are largely irrelevant



Best strategy for solving a problem using EC

- Choose **representation** to suit problem
- Choose variation operators to suit representation
- Choose selection operators, remembering that they only use fitness and so are independent of representation
- Establish the **baseline**
- Design your own method/variation
- Design comparative experiments between baseline and your own



Designing comparative experiments
(applying principled comparison of algorithms)

- Remember the **no free lunch theorem**

- For the problem to be solved decide the **budget** (number of generations / time) and also if it makes sense the **target value** (if we know what solution quality we would be satisfied with)

- What claim do you want to make? Formulate the null and alternate hypotheses (you will learn about these in this module)
- Run the evolutionary algorithm in the two different settings at least 30 times and record the results
- Apply Welch's t-test to obtain the **p-value**
- If the **p-value** is less than the desired significance level, reject the null hypothesis



Summary

- Evolutionary algorithms mimic Natural evolution
- The concepts of fitness, selection, variation, population are defined for artificial evolution
- The process of an evolutionary algorithm goes through generations consisting of:
  - selecting parents
  - producing offspring through recombination and mutation evaluating the offspring
  - selecting individuals for the next generation until a stopping criterion is met



Summary

- We considered
  - Selection
  - Variation operators
  - Experimental setup
  - Expected behaviour
- We are now ready to design and code an evolutionary algorithm for TSP



Recommended reading

- A. E. Eiben, J. E. Smith: Introduction to Evolutionary Computing, Chapters 2, 3, 4, 5 (chapter 1 also for background)

- The book's website
  `www.evolutionarycomputation.org`

